

Program Reasoning

8. Separation Logic

Kihong Heo



Motivation

- Problem of Hoare logic 1: unrelated facts

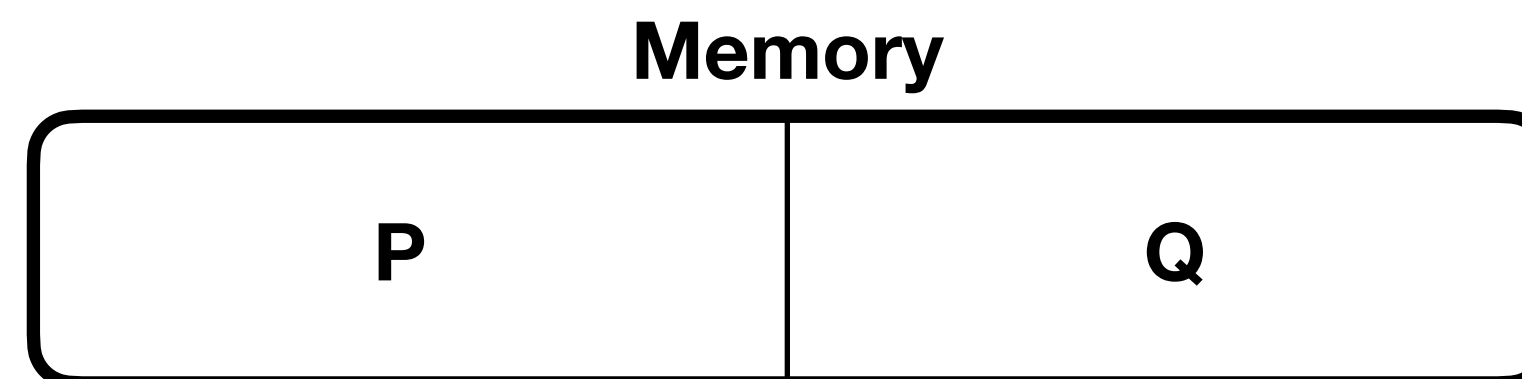
$$\{x = 3 \wedge y = 4 \wedge z = 9\} \ x := 5 \ \{x = 5 \wedge y = 4 \wedge z = 9\}$$

- Problem of Hoare logic 2: heap and alias

$$\{*p = 3 \wedge *q = 4\} \ *p := 5 \ \{*p = ? \wedge *q = ?\}$$

Separation Logic

- Extension of Hoare logic to handle advanced features of programming languages
 - E.g., memory, resources, concurrency, etc
- Key idea: separation
 - Split the heap and reason locally
- Example: two memory fragments are separated
 - Memory holds the whole property = left holds P and right holds Q
 - Enables local reasoning: P and Q are disjoint



Language

- The simple imperative as before + memory store, load, allocation, and free

$$\begin{aligned} S \rightarrow & \text{skip} \\ & | x := E \\ & | S; S \\ & | \text{if } E \text{ then } S \text{ else } S \\ & | \text{while } E \text{ do } S \\ & | x := [p] \\ & | [p] := E \\ & | p := \text{alloc}(E) \\ & | \text{free}(p) \end{aligned}$$

Operational Semantics

- A program state = a pair of stack and heap

$$State = Stack \times Heap$$

$$\sigma \in Stack = Var \rightarrow Val$$

$$h \in Heap = Addr \rightarrow Val$$

$$Val = \mathbb{Z} \cup Addr$$

- Big-step operational semantics

$$\frac{[[E]](\sigma) = v \quad p \notin Dom(h)}{\langle x := \text{alloc}(E), \sigma, h \rangle \Rightarrow \langle \sigma \{x \mapsto p\}, h \{p \mapsto v\} \rangle} \quad \frac{\sigma(p) = a \quad [[E]](\sigma) = v \quad a \in Dom(h)}{\langle [p] := E, \sigma, h \rangle \Rightarrow \langle \sigma, h \{a \mapsto v\} \rangle}$$
$$\frac{\sigma(p) = a \quad h(a) = v}{\langle x := [p], \sigma, h \rangle \Rightarrow \langle \sigma \{x \mapsto v\}, h \rangle} \quad \frac{\sigma(p) = a \quad a \in Dom(h)}{\langle \text{free}(p), \sigma, h \rangle \Rightarrow \langle \sigma, h \setminus \{a\} \rangle}$$

Assertions over Heap

- emp : The heap is empty
 - $(\sigma, h) \models \text{emp} \iff h = \emptyset$
- $x \mapsto v$ (singleton heap): The address stored in x points to a heap cell containing a value v
 - $(\sigma, h) \models x \mapsto v \iff h = \{\sigma(x) \mapsto v\}$
- $P * Q$ (separating conjunction): The heap can be split into disjoint parts satisfying P and Q
 - $\exists h_1, h_2. h = h_1 \uplus h_2 \wedge (\sigma, h_1) \models P \wedge (\sigma, h_2) \models Q$

Example (1)

```
    {emp}  
    p := alloc(1)  
    {p ↦ 1}  
    x := [p]  
    {p ↦ 1 ∧ x = 1}  
    q := p  
    {p ↦ 1 ∧ x = 1 ∧ p = q}  
    [q] := 2  
    {p ↦ 2 ∧ x = 1 ∧ p = q}  
    y := [p]  
    {p ↦ 2 ∧ x = 1 ∧ p = q ∧ y = 2}
```

Example (2)

```

    {emp}
    p := alloc(1)
    {p ↦ 1}
    r := alloc(10)
    {p ↦ 1 * r ↦ 10}
    x := [p]
    {x = 1 ∧ (p ↦ 1 * r ↦ 10)}
    q := p
    {x = 1 ∧ q = p ∧ (p ↦ 1 * r ↦ 10)}
    [q] := 2
    {x = 1 ∧ q = p ∧ (p ↦ 2 * r ↦ 10)}
    y := [r]
    {x = 1 ∧ q = p ∧ y = 10 ∧ (p ↦ 2 * r ↦ 10)}

```

Algebraic Laws of Separating Conjunction

- emp is a neutral element
- Structural laws of separating conjunction
 - Identity: $\text{emp} * P \iff P$
 - Commutativity: $P_1 * P_2 \iff P_2 * P_1$
 - Associativity: $(P_1 * P_2) * P_3 \iff P_1 * (P_2 * P_3)$

Separation Logic Rules (1)

- Rule for allocation

$$\frac{}{\{\text{emp}\} \ p := \text{alloc}(v) \ \{p \mapsto v\}}$$

- Rule for load (assume $x \neq p$)

$$\frac{}{\{p \mapsto v\} \ x := [p] \ \{p \mapsto v \ \wedge \ x = v\}}$$

- Rule for store

$$\frac{}{\{p \mapsto -\} \ [p] := v \ \{p \mapsto v\}}$$

- Rule for free

$$\frac{}{\{p \mapsto v\} \ \text{free}(p) \ \{\text{emp}\}}$$

Separation Logic Rules (2)

- Frame rule

$$\frac{\{P\} \ C \ \{Q\}}{\{P * R\} \ C \ \{Q * R\}} \quad \text{Mod}(C) \cap FV(R) = \emptyset$$

- $\text{Mod}(C)$: the set of stack variables potentially modified by command C
- $FV(R)$: the set of free stack variables appearing in assertion R
- Why the side condition?
 - C must not change stack variables on which the framed assertion R depends

Example (1)

$\{\text{emp}\} \ p := \text{alloc}(1); q := \text{alloc}(2) \ \{p \mapsto 1 * q \mapsto 2\}$

Example (2)

$$\frac{\{p \mapsto 10\} \ q := p \ \{p \mapsto 10\}}{\{p \mapsto 10 * q \mapsto 20\} \ q := p \ \{p \mapsto 10 * q \mapsto 20\}}$$

Application: Linked List

- Linked list as a recursive predicate

$$list(x) \triangleq (x = \text{null} \wedge \text{emp}) \vee \exists v, y. x \mapsto (v, y) * list(y)$$

- Assertion $x \mapsto (v, y)$: The address stored in x points to a heap cell containing a pair
 - value v
 - next y

List Reversal

```
p := null;
```

```
while x != null do
```

```
    y := [x.next];
```

```
    [x.next] := p;
```

```
    p := x;
```

```
    x := y;
```

List Reversal

```
{list(x)}  
p := null;  
  
while x != null do  
    // inv: list(p) * list(x)  
  
    y := [x.next];  
  
    [x.next] := p;  
  
    p := x;  
  
    x := y;  
  
{list(p)}
```

List Reversal

```
{list(x)}  
p := null;  
{list(p) * list(x)}  
while x != null do  
    // inv: list(p) * list(x)
```

```
y := [x.next];
```

```
[x.next] := p;
```

```
p := x;
```

```
x := y;
```

```
{list(p)}
```

$$\frac{\frac{\{emp\} \ p := null \ \{p = null \wedge emp\}}{\{emp\} \ p := null \ \{list(p)\}}}{\{list(x)\} \ p := null \ \{list(p) * list(x)\}}$$

List Reversal

```
{list(x)}  
p := null;  
{list(p) * list(x)}  
while x != null do  
  // inv: list(p) * list(x)  
  {list(p) * x ↦ (v, n) * list(n)}  
  y := [x.next];  
  
  [x.next] := p;  
  
  p := x;  
  
  x := y;  
  
{list(p)}
```

$$\text{list}(p) * \text{list}(x) \wedge x \neq \text{null}$$

$$\implies \exists v, n. \text{list}(p) * x \mapsto (v, n) * \text{list}(n)$$

Choose fresh logical variables v, n

List Reversal

```
 $\{list(x)\}$   
 $p := null;$   
 $\{list(p) * list(x)\}$   
while  $x \neq null$  do  
  // inv:  $list(p) * list(x)$   
   $\{list(p) * x \mapsto (v, n) * list(n)\}$   
   $y := [x.next];$   
   $\{list(p) * x \mapsto (v, n) * list(n) \wedge y = n\}$   
   $[x.next] := p;$ 
```

```
 $p := x;$   

$$\frac{\{x \mapsto (v, n)\} \ y := [x.next] \ \{x \mapsto (v, n) \wedge y = n\}}{\{list(p) * x \mapsto (v, n) * list(n)\} \ y := [x.next] \ \{list(p) * x \mapsto (v, n) * list(n) \wedge y = n\}}$$
  
 $x := y;$ 
```

```
 $\{list(p)\}$ 
```

List Reversal

```

{list(x)}
p := null;
{list(p) * list(x)}
while x != null do
  // inv: list(p) * list(x)
  {list(p) * x ↦ (v, n) * list(n)}
  y := [x.next];
  {list(p) * x ↦ (v, n) * list(n) ∧ y = n}
  [x.next] := p;
  {list(x) * list(n) ∧ y = n}
  p := x;

```

```

x := y;

```

$$\frac{\frac{\{x \mapsto (v, n)\} \ [x.next] := p \ \{x \mapsto (v, p)\}}{\{list(p) * x \mapsto (v, n) * list(n) \wedge y = n\} \ [x.next] := p \ \{list(p) * x \mapsto (v, p) * list(n) \wedge y = n\}}}{\{list(p) * x \mapsto (v, n) * list(n) \wedge y = n\} \ [x.next] := p \ \{list(x) * list(n) \wedge y = n\}}$$

```

{list(p)}

```

List Reversal

```
{list(x)}  
p := null;  
{list(p) * list(x)}  
while x != null do  
  // inv: list(p) * list(x)  
  {list(p) * x  $\mapsto$  (v, n) * list(n)}  
  y := [x.next];  
  {list(p) * x  $\mapsto$  (v, n) * list(n)  $\wedge$  y = n}  
  [x.next] := p;  
  {list(x) * list(n)  $\wedge$  y = n}  
  p := x;  
  {list(p) * list(n)  $\wedge$  y = n}  
  x := y;  
  
{list(p)}
```

List Reversal

```
{list(x)}  
p := null;  
{list(p) * list(x)}  
while x != null do  
  // inv: list(p) * list(x)  
  {list(p) * x ↦ (v, n) * list(n)}  
  y := [x.next];  
  {list(p) * x ↦ (v, n) * list(n) ∧ y = n}  
  [x.next] := p;  
  {list(x) * list(n) ∧ y = n}  
  p := x;  
  {list(p) * list(n) ∧ y = n}  
  x := y;  
  {list(p) * list(x)}  
{list(p)}
```

$$\begin{aligned} &list(p) * list(x) \wedge x = \text{null} \\ &\implies list(p) * \text{emp} \\ &\implies list(p) \end{aligned}$$

Summary

- Separation logic: an extension of Hoare logic for practical verification
- Local reasoning via separation
 - Frame rule: reason only about the heap that a command accesses
- Key application: reasoning over heap memory